Étude et analyse de modèles simulant une trajectoire circulaire

Manuel Rocca, Damian Kazberuk, Matteo Morbée, Romain Liefferinckx, Martin Gouverneur

April 29, 2026

Introduction

La modélisation de systèmes dynamiques implique un compromis entre fidélité physique, coût de calcul et interprétabilité. Ce travail étudie la simulation de la trajectoire d'une bille en mouvement circulaire sur un dispositif expérimental, un système simple mais suffisamment riche pour comparer différentes approches.

Deux familles de méthodes sont considérées. Les modèles physiques explicites (conique et de Laplace), résolus par intégration numérique. La seconde famille, quant à elle, repose sur l'apprentissage automatique (régression linéaire, MLP) approximent la dynamique à partir de données, sans hypothèses a priori, mais avec une interprétabilité réduite.

L'analyse se penche aussi sur les erreurs de mesure, de modélisation et numériques, ainsi que sur les propriétés de consistance, stabilité et conditionnement. L'objectif est d'évaluer les performances, les limites et le domaine de validité de ces approches du point de vue numérique.

Modèles physiques

La première classe de méthode de simulation utilise les équations fondamentales de la physique. La dynamique est réduite à la surface par la pente locale dz/dr, produisant une accélération radiale effective $a_c(r) = g\,|dz/dr|$ et une force de frottement $\vec{F}_f = -\mu_{\text{eff}}\, g\, \vec{v}/\|\vec{v}\|$. Les équations complètes figurent en Annexe.

Modèle conique. Pente constante α : $a_c = g\tan(\alpha)$. L'accélération radiale est uniforme, ce qui simplifie l'intégration mais ignore toute variation locale de courbure.

Modèle de Laplace. Pente en $K/(mgr^2)$, donnant $a_r = K/(mr^2)$. La dépendance en $1/r^2$ reproduit une dynamique non linéaire plus réaliste, au prix d'une singularité au centre nécessitant une régularisation pour $r < r_c$ (voir Annexe).

Intégration numérique

Afin de résoudre les équations différentielles, il est nécessaire de procéder à une intégration numérique du système. Elles sont résolues par quatre méthodes. Le choix d'un intégrateur résulte d'un compromis entre ordre de précision, conservation de l'énergie à long terme et coût par pas.

Les schémas sont définis avec (q_n, p_n) désignant position et quantité de mouvement à t_n :

Euler explicite (ordre 1, non-symplectique) :

$$q_{n+1} = q_n + \Delta t\, p_n/m, \qquad p_{n+1} = p_n + \Delta t\, F(q_n)$$

Euler semi-implicite (ordre 1, symplectique) :

$$p_{n+1} = p_n + \Delta t\, F(q_n), \qquad q_{n+1} = q_n + \Delta t\, p_{n+1}/m$$

Velocity-Verlet (ordre 2, symplectique) :

$$q_{n+1} = q_n + \Delta t\,\frac{p_n}{m} + \frac{\Delta t^2}{2m}F(q_n),$$
$$p_{n+1} = p_n + \frac{\Delta t}{2}\left[F(q_n) + F(q_{n+1})\right]$$

Runge-Kutta 4 (ordre 4, non-symplectique) : évaluation de quatre pentes $k_1, \ldots, k_4$ à des sous-pas intermédiaires, combinées selon $y_{n+1} = y_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$.

Sur le plan numérique, l'intégration repose sur des systèmes discrets à pas de temps fini, ce qui introduit des erreurs de troncature et d'arrondi. Les singularités possibles (en particulier au centre) sont évitées par une régularisation ou un rayon minimal afin de garantir la stabilité des simulations. Le tableau 1 synthétise les propriétés clés.

Modèle basé sur du machine learning (ML)

Une seconde classe de méthodes de simulation repose sur l'utilisation de techniques d'apprentissage automatique, visant à approximer directement la dynamique du système à partir de données. Contrairement aux approches déterministes fondées sur la modélisation explicite des lois physiques, ces méthodes s'inscrivent dans un cadre empirique, dans lequel la relation entrée-sortie est apprise à

Table 1: *Propriétés comparées des schémas d'intégration numérique.*

Méthode	Ordre	Symplectique	Stabilité ($\lambda = i\omega$)	Coût (éval. F)
Euler exp.	1	Non	Instable	1
Euler semi-imp.	1	Oui	Marginale	1
Velocity-Verlet	2	Oui	Conditionnelle	2
RK4	4	Non	$\lvert\lambda\Delta t\rvert \lesssim 2.83$	4

partir d'un ensemble fini d'observations. Dans cette étude, les données générées par les simulations physiques sont utilisées comme ensemble d'entraînement. L'objectif du ML est d'apprendre une fonction

$$f : \mathbb{R}^4 \to \mathbb{R}^{2N}$$

telle que:

$$\mathbf{y} = f(\mathbf{x}),$$

où $\mathbf{x}$ représente les conditions initiales du système et $\mathbf{y}$ la trajectoire correspondante.

Plus précisément, le vecteur d'entrée est défini par:

$$\mathbf{x} = (x_0, y_0, v_{x0}, v_{y0}) \in \mathbb{R}^4,$$

où x_0 et y_0 désignent les coordonnées initiales de la bille, et v_{x0}, v_{y0} les composantes initiales de sa vitesse.

La sortie correspond à une discrétisation temporelle de la trajectoire:

$$\mathbf{y} = (x_1, y_1, x_2, y_2, \ldots, x_N, y_N) \in \mathbb{R}^{2N},$$

où N désigne le nombre de pas de temps considérés. Cette formulation permet d'exprimer la dynamique du système sous la forme d'une application entre vecteurs d'entrée et de sortie.

Deux modèles ont été considérés pour approximer la fonction f: la régression linéaire (LR) et le perceptron multicouche (MLP).

Regression linéaire La régression linéaire constitue un modèle paramétrique dans lequel la sortie est exprimée comme une combinaison affine des variables d'entrée [1]. Dans ce cas, elle est utilisée pour approximer la fonction $f : \mathbb{R}^4 \to \mathbb{R}^{2N}$ reliant les conditions initiales à la trajectoire de la bille.

Plus précisément, le modèle s'écrit:

$$\hat{\mathbf{y}} = \mathbf{W}\mathbf{x} + \mathbf{b},$$

où $\mathbf{W} \in \mathbb{R}^{2N\times 4}$ est une matrice de paramètres et $\mathbf{b} \in \mathbb{R}^{2N}$ un vecteur de biais.

Chaque composante de la trajectoire prédite est ainsi modélisée comme une combinaison linéaire des conditions initiales. Cette approche revient à supposer que la dynamique du système peut être approximée globalement par une transformation affine.

Toutefois, un tel modèle ne permet pas de capturer des dépendances temporelles explicites entre les différents instants de la trajectoire, ni des comportements non linéaires. Par conséquent, sa capacité de modélisation reste limitée pour des dynamiques complexes.

Multicouche perceptron Afin de modéliser des comportements plus complexes, un perceptron multicouche a également été utilisé. Dans le cas d'une architecture à une couche cachée, la fonction f s'écrit:

$$f(\mathbf{x}) = \mathbf{W}^{(2)}\,\sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)},$$

où σ est une fonction d'activation non linéaire. σ est représentée par la fonction ReLU (Rectified Linear Unit), définie par $\sigma(z) = \max(0, z)$ [1].

Les paramètres du modèle, regroupés dans θ, sont estimés en minimisant une fonction de coût, généralement choisie comme l'erreur quadratique moyenne (MSE):

$$\mathcal{L}(\theta) = \frac{1}{M}\sum_{i=1}^{M}\left\|\mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)}\right\|_2^2,$$

où M désigne le nombre d'échantillons d'entraînement. Pour chaque exemple i, $\mathbf{y}^{(i)}$ correspond à la trajectoire réelle associée aux conditions initiales $\mathbf{x}^{(i)}$, et $\hat{\mathbf{y}}^{(i)} = f_\theta(\mathbf{x}^{(i)})$ la prédiction du modèle.

Validité des modèles de ML

Afin d'évaluer la capacité de généralisation des modèles d'apprentissage automatique, une procédure de validation croisée à K plis (*K-fold cross-validation*) a été mise en œuvre. Cette méthode consiste à partitionner aléatoirement l'ensemble des données d'entraînement en K sous-ensembles de taille égale. À chaque itération, l'un des K plis est réservé à la validation tandis que les $K-1$ plis restants servent à l'entraînement. La procédure est répétée K fois de sorte que chaque pli joue exactement une fois le rôle d'ensemble de validation. La performance finale est estimée par la moyenne des erreurs obtenues sur chacun des plis [2] [1].

Dans cette étude, $K = 5$ a été retenu. Le MLP est entraîné sur 50 epochs à chaque pli. La métrique d'évaluation choisie est l'erreur quadratique moyenne sur le rayon (*RMSE rayon*), définie par:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(r_i - \hat{r}_i)^2}$$

où r_i désigne la valeur réelle du rayon à l'instant i et $\hat{r}_i$ la valeur prédite par le modèle, évalués sur un ensemble de test indépendant de 64 trajectoires générées séparément.

Les résultats obtenus sont résumés dans le tableau suivant:

Table 2: *Résultats de la validation croisée 5-fold pour les modèles linéaire et MLP, entraînés sur 90 000 expériences.*

Modèle	RMSE rayon moyen μ	Écart-type σ
Linéaire	8.41×10^{-3} m	1×10^{-5} m
MLP	5.45×10^{-3} m	1.36×10^{-3} m

Plusieurs observations peuvent être tirées de ces résultats. D'une part, le MLP affiche une RMSE moyenne inférieure de 35 % à celle du modèle linéaire, ce qui témoigne d'une meilleure capacité à capturer la dynamique non linéaire du système. D'autre part, l'écart-type associé au modèle linéaire est négligeable ($\sigma \approx 10^{-5}$ m), traduisant une très grande stabilité entre les plis: le modèle produit systématiquement la même performance quelle que soit la partition des données. Ce comportement est caractéristique d'un modèle à faible variance mais à biais élevé, dont la capacité d'expression est trop limitée pour s'adapter à la complexité des données.

Le MLP présente quant à lui un écart-type plus marqué ($\sigma = 1.36 \times 10^{-3}$ m), révélant une sensibilité accrue à la composition de l'ensemble d'entraînement. Cette variabilité est néanmoins cohérente avec la nature d'un modèle à plus haute capacité, dont les performances dépendent davantage de la distribution des données disponibles. Il est important de noter que même dans le cas le moins favorable observé (RMSE $= 7.48 \times 10^{-3}$ m), le MLP demeure compétitif face au modèle linéaire, et surpasse ce dernier dans tous les plis. La validation croisée confirme ainsi que la supériorité du MLP est robuste et non due à un artefact de partitionnement particulier.

Ces résultats sont illustrés en annexe: la figure 1 présente les trajectoires prédites par chaque modèle en fonction de la taille du jeu de données, tandis que la figure 2 détaille l'évolution de la RMSE par pli pour les deux modèles.

Erreurs de modélisation

La catégorie des erreurs de modélisation se décline en deux types d'erreurs.

Le premier type, les erreurs de mesure qui se produisent dans le cadre physique de l'expérience. En effet, ne disposant pas d'outils de mesure parfaits, des imprécisions numériques de l'ordre du millimètre ont été introduites lors des mesures des éléments de l'expérience, comme par exemple la largeur du dispositif.

Le second regroupe les erreurs de modélisation du phénomène; des erreurs qui apparaissent lors de simplifications, ou des hypothèses dans le modèle pour le rendre plus facile à comprendre ou à résoudre. Un exemple d'hypothèse émise dans le cadre de l'expérience concerne la pression de l'air, supposée sans impact sur la trajectoire de la balle.

Calcul numérique

Erreurs numériques

Dans le cadre d'une approche scientifique, il est important de prendre en compte tous les paramètres pouvant entrer en jeu et potentiellement perturber et modifier l'expérience. Les erreurs de modélisation ont déjà été abordées, toutefois, elles ne sont pas les seules à intervenir. En particulier, il faut considérer les erreurs numériques qui apparaissent lorsque l'ordinateur effectue des calculs [3, p. 16].

Erreurs de troncature Les modèles itératifs à pas discret décrits ici sont infinis. Il est donc impossible d'obtenir une solution exacte en un temps fini même en faisant abstraction de tout autre type d'erreur pouvant entrer en jeu. Afin d'obtenir un résultat, il faut établir une variable *n_steps* qui limite le nombre d'itération de manière à obtenir un bon équilibre entre temps d'exécution et précision du résultat.

Erreurs d'arrondi Les erreurs d'arrondi sont des erreurs causées par la représentation finie des nombres en machine dans les registres du processeur (généralement 32 ou 64 bits). L'ordinateur ne peut pas représenter tous les nombres réels avec une précision infinie, ce qui crée alors une approximation.

Pour définir formellement ce type d'erreur, notons par $\mathbb{F}(b, t, L, U)$ l'ensemble des nombres réels qui sont représentés par une notation à virgule flottante en base b, comportant t chiffres significatifs et dont l'exposant varie dans l'intervalle $[L, U]$. Considérons un ordinateur utilisant cette notation $\mathbb{F}$. Soit $x \in \mathbb{R}$ un nombre réel quelconque et $fl(x) \in \mathbb{F}$ la représentation machine à virgule flottante de x. Désignons par ε_x son erreur relative

$$\varepsilon_x = \frac{|fl(x) - x|}{|x|} \tag{1}$$

Evaluons la borne supérieure de l'erreur relative d'arrondi. Il est évident que cette quantité ne peut

dépasser la moitié de la distance relative entre les deux nombres consécutifs à virgule flottante $x_i \in \mathbb{F}(b,t,L,U)$ et $x_{i+1} \in \mathbb{F}(b,t,L,U)$ tels que

$$x_i \leq x \leq x_{i+1} \tag{2}$$

Il vient alors que

$$\varepsilon_x \equiv |\rho_x| \leq \frac{1}{2}\left|\frac{x_{i+1}-x_i}{x}\right| \leq \frac{1}{2}b^{1-t} = \frac{1}{2}\epsilon = u, \tag{3}$$

où u est dite la précision machine.

Avec la notion d'écart relatif définie telle que

$$\rho_x = \frac{\hat{x}-x}{x} \tag{4}$$

il en résulte

$$\hat{x} = fl(x) = x(1+\rho_x), \tag{5}$$

où $\varepsilon_x = |\rho_x| \leq u$ est dite l'erreur d'arrondi [3].

Il est évident que ce type d'erreur apparaît également dans les simulations.

Erreurs de propagation et génération Les erreurs de propagation et de génération sont des erreurs qui apparaissent lorsque les erreurs numériques se propagent à travers les calculs. Les erreurs de génération se produisent à chaque itération, étape des calculs et les erreur de propagation regroupent toutes les erreurs de génération jusqu'à l'étape courante.

Analyse numérique du problème

Les sous-sections suivantes portent sur les notions de consistance, de stabilité et de conditionnement, afin d'analyser les propriétés des modèles considérés. Ces analyses permettent de prédire certains comportements des modèles lors des simulations.

Définissons d'abord la notion d'algorithme numérique. Étant donné un problème bien posé $F(x,d) = 0$, l'algorithme numérique de résolution du problème F est défini comme une suite de problèmes approchés

$$F_1\left(x^{(1)},d^{(1)}\right),\ F_2\left(x^{(2)},d^{(2)}\right),\ \ldots,\ F_n\left(x^{(n)},d^{(n)}\right) \tag{6}$$

dépendant d'un paramètre n, où $x^{(n)}$ est la solution du sous-problème F_n [3].

Consistance

La consistance traduit la capacité d'un modèle numérique à tendre vers les lois pures de la physique lorsque le pas de temps s'affine. Formellement [3], un schéma $F_n(x^{(n)},d^{(n)})$ est consistant si la solution exacte x vérifie le modèle limite:

$$\lim_{n\to\infty} F_n(x,d^{(n)}) = F(x,d) = 0 \tag{7}$$

En pratique, pour une trajectoire $y' = f(t,y)$, la méthode d'Euler (explicite ou semi-implicite) admet le développement de Taylor suivant [3]:

$$y(t_{n+1}) = y(t_n) + \Delta t f\big(t_n, y(t_n)\big) + \mathcal{O}(\Delta t^2) \tag{8}$$

L'erreur locale de troncature par pas de temps est $\tau_{n+1} = \mathcal{O}(\Delta t)$. Ces méthodes sont donc consistantes à l'ordre 1. À l'inverse, l'approche de Runge-Kutta (RK4) annule analytiquement les termes d'erreur d'ordre inférieur, offrant $\tau_{n+1} = \mathcal{O}(\Delta t^4)$, soit une très haute consistance polynomiale.

Stabilité

La stabilité garantit que les perturbations inévitables (comme les erreurs d'arrondi) ne s'amplifient pas exponentiellement. Un algorithme est numérique stable si, pour tout $\epsilon > 0$, il existe $\eta > 0$ tel que toute perturbation initiale d'amplitude η affecte la solution finale de moins de ϵ.

Pour l'analyser, l'équation de test linéaire est couramment utilisée $y' = \lambda y$ (avec $\lambda \in \mathbb{C}$) [3]. Pour la méthode d'Euler explicite, sa récurrence dicte $y_{n+1} = G \cdot y_n$, avec $G = (1+\lambda\Delta t)$ [4]. La stabilité exige impérativement $|G| \leq 1$. Cependant, pour la trajectoire purement inertielle (système conservatif), λ est imaginaire pur ($\lambda = i\omega$). Le facteur d'amplification devient:

$$|G| = |1+i\omega\Delta t| = \sqrt{1+\omega^2\Delta t^2} > 1 \tag{9}$$

La méthode d'Euler explicite amplifie artificiellement l'énergie du système oscillatoire et diverge de son orbite. En structurant l'intégration différemment, la méthode d'Euler semi-implicite met à jour la position depuis la vitesse future. Cette conception *symplectique* préserve le volume de l'espace des phases et assure une grande stabilité à long terme pour les orbites.

Convergence

Le théorème d'équivalence de Lax [3] stipule logiquement que pour un problème bien posé, la consistance couplée à la stabilité assure la convergence globale du modèle discret vers la vérité continue.

Toutefois, ce théorème ne s'extrapole pas aux réseaux de neurones (MLP, LR). Entraînés de manière isolée pour lisser l'étape $n \to n+1$, la régression est aveugle aux variations sur temps long. Intégrée en boucle auto-régressive (*step-by-step*), ses légères imprécisions déplacent l'état évalué hors du domaine vu à l'apprentissage (un *distribution shift*). Ces erreurs alimentent exponentiellement les futures imprécisions, déclenchant sur la durée une déviation divergente similaire à de l'instabilité numérique classique.

Conditionnement

Contrairement à la stabilité qui évalue l'algorithme choisi, le conditionnement diagnostique la susceptibilité intrinsèque du *problème analytique* en lui-même. Pour une évolution de type $y' = f(t, y)$, la variation $\delta y(t)$ issue d'une erreur d'entrée se dilate proportionnellement à la matrice Jacobienne $J = \frac{\partial f}{\partial y}$:

$$\delta y' = J\delta y \implies \|\delta y(t)\| \propto \exp\left(\int \|J\| dt\right) \tag{10}$$

Dans le système circulaire abordé ici, les valeurs propres de la Jacobienne ont une composante réelle négligeable: l'erreur n'y grimpe pas de façon irrémédiable, et ce problème est caractérisé comme globalement bien conditionné [5]. Toutefois, la manipulation informatique y imbrique de permanentes erreurs d'arrondi (dues à la précision machine limitée $u = \frac{1}{2}b^{1-t}$). Ce contexte bien conditionné n'évitera ainsi pas la perte d'exactitude si un schéma divergent (comme Euler explicite) est forcé sur la simulation.

Comparaison des méthodes d'approximation numérique

Plusieurs méthodes d'approximation numérique peuvent être utilisées pour résoudre les équations différentielles ordinaires associées aux différents modèles étudiés. Le tableau suivant présente, pour chaque méthode et chaque modèle, l'erreur géométrique normalisée mesurant l'écart entre la trajectoire simulée et la trajectoire réelle.

Table 3: *Erreur géométrique normalisée entre la trajectoire simulée et la trajectoire réelle pour chaque combinaison de modèle et de méthode numérique.*

Méthode	Conique	Laplace	Laplace R_C
Euler exp.	0.7681	0.7682	0.7680
Euler semi-imp.	0.7681	0.7682	0.7680
RK4	0.7682	0.7682	0.7680

Conclusion et discussion

Dans le cadre de ce travail, la comparaison des différents modèles de simulation ne peut se limiter à un unique critère de performance. En effet, les approches considérées présentent des caractéristiques hétérogènes, tant du point de vue de la fidélité physique que des propriétés numériques ou de leur mise en œuvre. Il apparaît ainsi nécessaire d'introduire un cadre d'analyse permettant de confronter ces modèles selon plusieurs dimensions pertinentes.

À cette fin, une démarche d'évaluation multicritère est adoptée. Celle-ci repose sur l'identification de quatre axes principaux:

- la précision des résultats, évaluée à travers les erreurs numériques (40%)
- les hypothèses simplifiantes considérées et leur quantité (30%)
- le temps requis pour effectuer les simulations (10%)
- l'accessibilité du modèle, incluant à la fois sa simplicité de mise en œuvre au niveau technique et sa capacité à être interprété ou vulgarisé (20%)

En attribuant les notes correspondantes à chaque modèle selon ces critères, les résultats obtenus sont les suivants:

Une telle approche permet de construire un indicateur synthétique facilitant la comparaison globale des modèles, tout en conservant une lecture structurée des compromis qu'ils impliquent. Elle s'inscrit dans une logique d'aide à la décision plutôt que dans une volonté d'établir un classement absolu.

soulignons toutefois que cette pondération repose en partie sur des choix méthodologiques propres à ce travail subjectifs. Elle a pour but de regrouper tous les aspects rencontrés au cours de ce travail, notamment l'aspect de vulgarisation qui représentait un aspect majeur lors de la présentation au Printemps des Sciences. Dans ce sens, l'évaluation proposée doit être comprise comme qualitative et indicative: elle vise à structurer l'analyse et à expliciter les compromis entre modèles, sans prétendre à une objectivité stricte ni à une généralisation indépendante du contexte d'étude.

Annexes

Équations physiques

Cadre physique commun Les deux modèles physiques partagent la même description cinématique et reposent sur une réduction de la dynamique à la surface. La géométrie est entièrement encodée par la pente locale dz/dr, ce qui permet d'écrire une accélération radiale effective :

$$a_c(r) = g \left| \frac{dz}{dr} \right|$$

Le frottement est modélisé par une force opposée à la vitesse, identique quelle que soit la direction du mouvement, et de norme proportionnelle à g :

$$\vec{F}_f = -\mu_{\text{eff}}\, g\, \frac{\vec{v}}{\|\vec{v}\|}$$

Ce choix permet de décrire une dissipation globale simple, sans distinguer de direction privilégiée ni introduire explicitement la réaction normale.

Cadre physique du cône Le modèle conique suppose une pente constante α :

$$\frac{dz}{dr} = \tan(\alpha)$$

L'intégration donne une hauteur linéaire en r :

$$z(r) = \tan(\alpha)\,(r - r_{\min})$$

L'accélération radiale devient alors constante :

$$a_c = g \tan(\alpha)$$

Cadre physique de Laplace Le modèle de Laplace introduit une pente variable, inspirée d'un champ central :

$$\frac{dz}{dr} = \frac{K}{mgr^2}$$

L'intégration conduit à :

$$z(r) = -\frac{K}{mgr} + C$$

D'où une accélération radiale :

$$a_r = g \left| \frac{dz}{dr} \right| = \frac{K}{mr^2}$$

Cette dépendance en $1/r^2$ implique une dynamique fortement non linéaire.

Régularisation Pour éviter la divergence au centre, une pente régularisée est utilisée :

$$\frac{dz}{dr} = \begin{cases} \frac{K}{mgr^2} & \text{si } r \geq r_c \\ \frac{K}{mgr_c^2} & \text{si } r < r_c \end{cases}$$

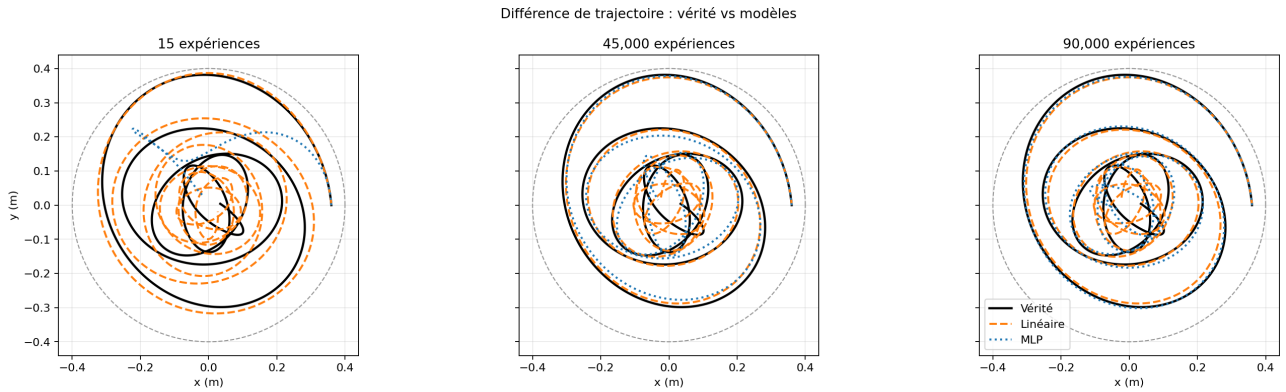

Figure 1: *Différence entre la vraie trajectoire et les modèless linéaire et MLP pour trois tailles de jeu de données (15, 45000, 90000).*

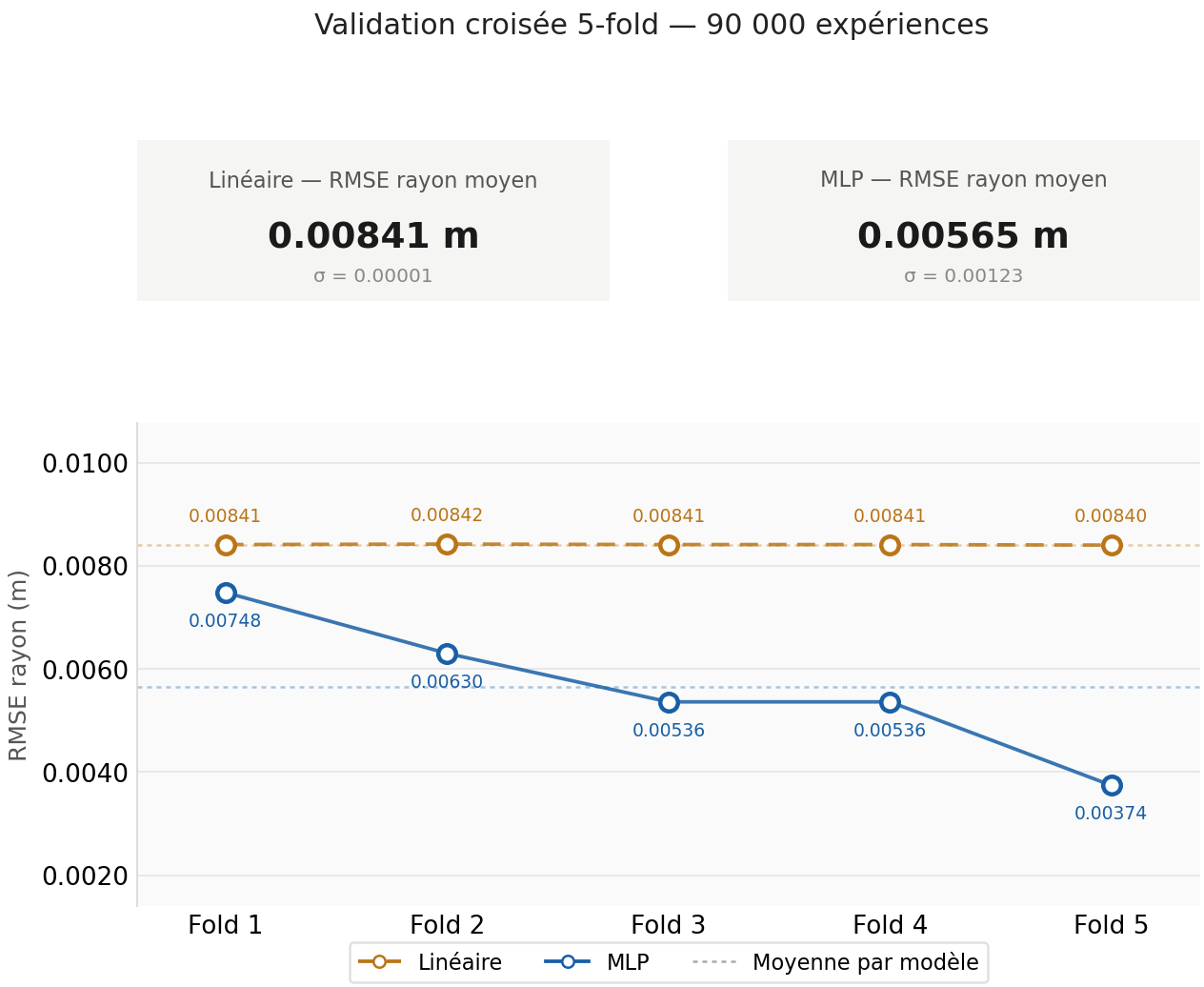

Figure 2: *Cross-validation avec K-fold (K=5) pour les modèles regression linéaire et MLP.*

References

[1] Université Libre de Bruxelles. *Statistical Foundations of Machine Learning*. INFO-F422. 2025. URL: https://ulb-mlg.github.io/INFO-F422/practicals/_build/html/03.html.

[2] GeeksforGeeks. *Cross Validation in Machine Learning*. Sept. 2025. URL: https://www.geeksforgeeks.org/machine-learning/cross-validation-machine-learning/.

[3] Maarten Jansen et al. *SYLLABUS Cours de Calcul Formel et Numérique*. INFO-F-205. Belgique, Feb. 2022.

[4] John C. Strikwerda. *Finite Difference Schemes and Partial Differential Equations*. 2nd. Philadelphia, PA: SIAM, 2004. ISBN: 0898715679.

[5] Gianluca Bontempi. *SYLLABUS Cours de Modélisation et Simulation*. INFO-F-305. Belgique, July 2024.